Perception-Driven Window Racing on a Quadrotor

Anshul Raje
Dept. of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA 01609
Email: arraje@wpi.edu

Imaan Haider
Dept. of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA 01609
Email: ihaider@wpi.edu

Zac Serocki
Dept. of Robotics Engineering
Worcester Polytechnic Institute
Worcester, MA 01609
Email: zdserocki@wpi.edu

***Abstract*—This project presents a perception-driven autonomy stack for flying a Crazyflie through a sequence of windows. A lightweight window-corner detector is trained on synthetic images with varied viewpoints, lighting, backgrounds, and image effects, and evaluated on Gaussian-splat views of the flight area. The detected inner corners are used to estimate each window's center and normal through planar pose estimation while RANSAC rejects inconsistent estimates across frames. The policy switches between searching for windows, planning and following paths, and flying through each opening. RRT* planning and quintic polynomial segments form smooth trajectories for a velocity controller to follow while adjusting the drone's heading. The stack is developed in simulation and evaluated in hardware-in-the-loop flights with the real drone. The drone completed the course in four of eight flights, in 9.25 s on average, with a median control tick of about 20 ms against the 66 ms frame budget. All four failures came on the climb to the last window, where the drone's altitude lagged the command.**

I. Problem Statement

The goal of this project is to build an autonomy stack that flies a Crazyflie through three windows in order. The drone must find each window, estimate the center of its opening and the normal vector through it, and plan a path that passes through without hitting the frame or its supports. The windows are placed at different heights and turned relative to the course, so the drone must adjust both its position and heading throughout.

During lab testing, the drone flies under motion capture, while its front-facing RGB-D images are rendered from a Gaussian splat at its current pose. The drone's position is available from Vicon, but the window locations must come from the perception stack. The detector is trained only on synthetic images that we generate. The provided splats are used for testing. Rendering, detection, planning, and control share a 15 Hz loop, giving about 66 ms per frame. The stack is first tested in simulation, and the same policy is then run on the real drone, with the goal of passing all three windows while avoiding collisions and completing the course as quickly as possible.

II. Synthetic Dataset Generation

The window detector is trained only on synthetic images that we generate ourselves. Renders of the Gaussian splat are used for testing alone and never for training. A single script builds the scene from scratch through the Blender Python API [1], renders zero to three windows, composites them over a background and writes the labels in the Ultralytics pose format [2]. Every label is computed from the known scene geometry, so no image is labeled by hand. The randomized parameters follow the idea of domain randomization [3] and are listed in Table I.

A. Window Model

The window texture is made from the print file in the starter pack. We crop it along the outer cut line to the print as it looks when mounted, keeping the white margin, the crimson frame and the white band around the opening. Both black cut lines are removed and the opening is made transparent. The crop is pixel exact, so the aspect ratio of the real print is kept. A second texture paints over the WPI logos and the PeAR text so the network cannot rely on them. It is used for 20% of the windows. Both are shown in Fig. 1.

In Blender each window is a flat mesh of four quads around the opening. The opening is therefore real geometry, so the background and farther windows show through it, and occlusion between windows can be tested exactly by ray casting. Each window gets a random surface roughness.

B. Camera and Scene Randomization

The Blender camera matches the drone camera in resolution and field of view, so images and labels need no resizing. Each image holds zero to three windows. Every window is placed at a random distance within the range of the depth sensor, somewhere in the view, and turned by a random yaw, pitch

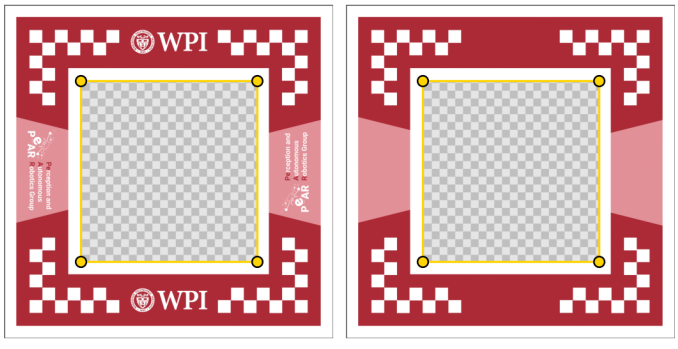


Fig. 1: Window textures with and without logos, cut from the starter print. The opening is transparent and the four keypoints are its inner corners.

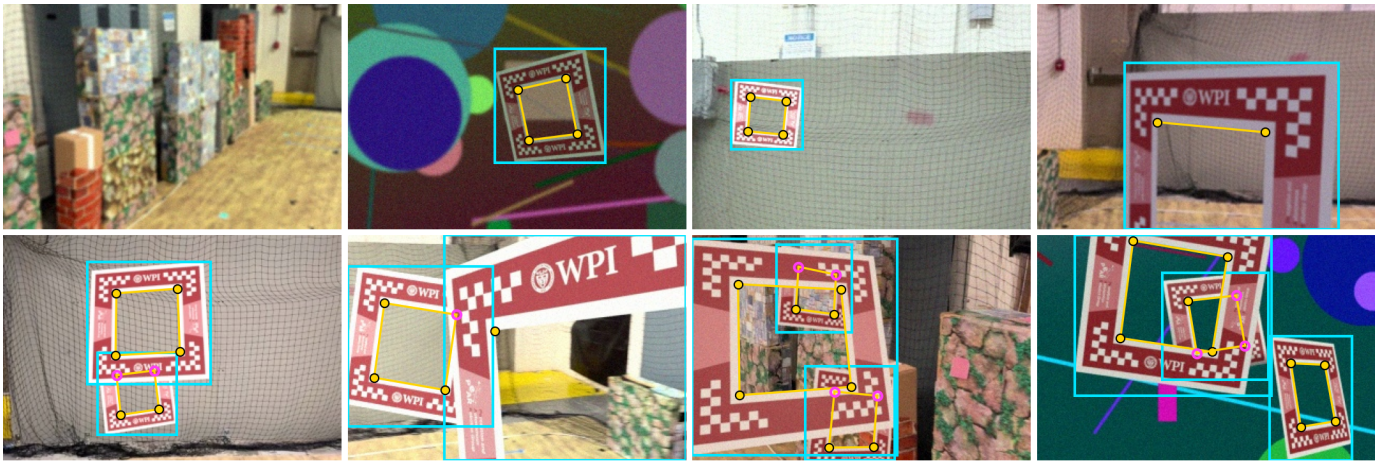


Fig. 2: Samples from the synthetic test split with their labels. Cyan boxes cover the visible print. Filled yellow points are visible keypoints and hollow magenta points are keypoints hidden behind a nearer window. Keypoints outside the image are not drawn. The top row shows video and procedural backgrounds with no window, one window and a window cut off by the image edge. The bottom row shows two and three windows with occlusion.

TABLE I: Randomized Parameters per Image

Parameter	Range
Windows per image	0, 1, 2, 3 with $p = 0.1, 0.4, 0.3, 0.2$
Distance	$\mathcal{U}(0.5, 4.0)$ m
Center in view	central 80% of each axis
Yaw, pitch, roll	$\pm 45°$, $\pm 15°$, $\pm 10°$
Depth gap if overlapping	≥ 0.3 m
Visible print fraction	≥ 0.2
Texture	logo 0.8, no logo 0.2
Roughness	$\mathcal{U}(0.3, 0.9)$
Sun strength	log-uniform in $[1, 3]$
Sun angular size	$\mathcal{U}(0.05, 0.6)$ rad
Ambient strength	$\mathcal{U}(0.05, 0.5)$
Background	video 0.5, procedural 0.5
Background crop	40% to 100% of the image, flip 0.5
Channel gain, exposure	$\mathcal{U}(0.9, 1.1)$, $\mathcal{U}(0.85, 1.15)$
Gamma	$\mathcal{U}(0.8, 1.25)$
Blur	$p = 0.3$, 3 or 5 tap binomial
Gaussian noise σ	$\mathcal{U}(0, 0.03)$

and roll. The yaw range covers the turn of the real windows plus the heading error of the drone.

Layouts that could not happen in the real course are drawn again. Windows that overlap in the image must be far enough apart in depth that they never pass through each other, and every window must have at least a fifth of its print in view and not hidden behind another window. The scene is lit by a sun lamp with a random direction, softness, strength and color, plus random ambient light.

C. Backgrounds and Compositing

The windows are rendered on a transparent background and pasted over a background image. Half of the backgrounds are frames from a video we recorded of the flight area with no windows in view. The other half are random procedural images of shapes and gradients, which keep the network from relying on the look of the flight area. Finally, random camera effects are applied to the whole image. These are white balance, exposure, gamma, blur and noise.

D. Annotations

Each window in an image gets one label made of a class, a bounding box and four keypoints with a visibility flag each. The keypoints are the four corners of the opening, projected into the image with the camera intrinsics. We use the opening because it is the part the drone flies through.

The print looks the same when mirrored, so a network cannot tell which physical corner it is looking at. The keypoints are therefore ordered by where they land in the image, as top left, top right, bottom right and bottom left. They are sorted by angle around their centroid, starting at the corner with the smallest sum of its two pixel coordinates. With this order a horizontal flip maps the keypoints onto each other by the permutation $[1, 0, 3, 2]$, which is passed to the training augmentation.

The visibility flag is 0 when a corner is outside the image, and its coordinates are then ignored in the loss. It is 1 when the corner is inside the image but hidden behind a nearer window, found by casting a ray from the camera to the corner. It is 2 when the corner is visible. Hidden corners are still supervised, so the network learns to place them from the visible part of the frame.

The bounding box is drawn around the whole print, not just the opening. When a window is cut off by the image edge, the box is drawn only around the part of the print inside the image [4], so it stops at the edge instead of covering pixels the camera never sees. A metadata file also stores the true pose of each window in the camera frame, for later checks of the window pose estimate.

E. Dataset

The final dataset has 25,000 images. Each image is assigned to the training, validation or test split with probabilities 0.8, 0.1 and 0.1 when it is made. Every image is its own render,

so nothing leaks between splits. Samples with their labels are shown in Fig. 2.

III. Window Detection

A. Network

We detect windows with YOLO26n-pose [5], the smallest pose estimation model of the YOLO26 family. For every window in the image it predicts a score, a bounding box and the four keypoints defined in Section II, each with a confidence that the corner lies inside the image. Corners hidden behind a nearer window count as inside during training, so this confidence does not tell whether a corner is hidden. Instance segmentation would also separate the windows, but it gives a mask of each window. The corners of the opening would then have to be found from that mask in post-processing, for example by fitting lines to the inner edge of the frame, which gets unreliable when the window is cut off by the image edge or partly hidden. A keypoint detector predicts these corners directly. They are exactly the points the drone has to fly through, and each set of corners already belongs to one window.

The model is a small convolutional network that fits the CPU budget of the flight computer. YOLO26 is also trained end to end without non-maximum suppression. Each window gets a single prediction, so decoding the output only needs a score threshold.

B. Training

The network starts from weights pretrained on the COCO keypoint dataset [6] and is fine-tuned on our training split with Ultralytics [2]. On top of the randomization in the data, Ultralytics applies its own augmentation while training. We keep it milder than its defaults, which are tuned for COCO. Color jitter keeps the hue almost fixed so the crimson of the print stays crimson. Images are shifted, scaled and flipped horizontally, and the flip swaps the keypoints with the permutation from Section II. Rotation is off because the data already rolls the windows. Half of the batches are mosaics that tile four images into one, which shrinks the windows and creates new layouts. Mosaics are turned off for the last 10 epochs so the network ends on plain images.

Training ran for 100 epochs with a batch size of 64 on one NVIDIA A30 GPU of the WPI Turing cluster and took about 1.5 hours. The checkpoint with the best validation score is kept.

C. Inference

The trained network is exported to ONNX and run with ONNX Runtime on the CPU, so the flight code needs no deep learning framework. The camera image of 320×213 pixels is padded with gray rows to 320×224, since the network needs both sides to be multiples of 32. The padding matches the one used in training. The network returns a fixed list of candidate detections sorted by score. We keep those with a score above 0.5 and shift their coordinates back by the padding, which gives the box and the four corners of every window in the pixels of the camera image.

IV. Pose Estimation

A. World Location

To place each gate in the world frame, we estimate the pose of its opening from the four corner detections produced by the window detector. The detector returns the image coordinates of the inner corners of the opening (top-left, top-right, bottom-right, bottom-left) along with a visibility score for each corner. We only attempt pose estimation when all four corners are visible (vis > 0.5). A partially cut-off window gives a poorly constrained solution, so in that case the drone turns to bring the window fully into view first.

The opening is a square of known size, $0.47 \times 0.47\,\mathrm{m}$. We define its corners in a gate-centered object frame lying in the plane $z = 0$:

$$\mathbf{P}_i \in \left\{ \left(\pm\tfrac{w}{2}, \pm\tfrac{h}{2}, 0 \right) \right\}, \qquad w = h = 0.47\,\mathrm{m}. \tag{1}$$

Given these four 3D–2D correspondences and the camera intrinsics $\mathbf{K}$, we solve the Perspective-n-Point (PnP) problem for the rotation $\mathbf{R}$ and translation $\mathbf{t}$ that map the object frame into the camera frame:

$$s_i \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \mathbf{K} \left(\mathbf{R}\mathbf{P}_i + \mathbf{t} \right). \tag{2}$$

Because all four points lie in one plane, we use OpenCV's IPPE solver [7]. It is designed for planar targets and solves the four-point case exactly. The intrinsics come from a pinhole model with the principal point at the image center and focal length $f = \frac{W}{2} \big/ \tan\!\left(\frac{\mathrm{FOV}_x}{2}\right)$, where $\mathrm{FOV}_x \approx 60.66°$.

The translation $\mathbf{t}$ is the center of the opening in the camera frame. The third column of $\mathbf{R}$ is the opening's normal, $\mathbf{n}$. A plane's normal has no preferred sign, so we flip $\mathbf{n}$ when needed to make $\mathbf{n} \cdot \mathbf{t} > 0$. The normal then always points through the window, away from the drone, which is the direction the drone must fly.

To move these estimates into the world frame, we first convert from the camera convention (x right, y down, z forward) to the drone's body frame (x forward, y left, z up). We then rotate by the drone's yaw ψ and add its position $\mathbf{p}$:

$$\mathbf{c}_w = \mathbf{p} + \mathbf{R}_z(\psi)\, \mathbf{c}_b, \qquad \mathbf{n}_w = \mathbf{R}_z(\psi)\, \mathbf{n}_b. \tag{3}$$

We ignore roll and pitch because they stay within a few degrees during flight. The gates stand upright, so we set the vertical component of the world normal to zero and renormalize. This removes the tilt error that PnP may introduce into the normal.

B. Filtering

A single PnP estimate is noisy, and the noise grows with distance. When the gate is far away, a pixel of corner error is a large fraction of the opening's apparent size. We observed that PnP underestimated the range by roughly 14 cm beyond 2 m, compared with about 4 cm within 1.5 m. Occasional bad corner detections also produce estimates that are far off in both position and orientation. For these reasons we gather many sightings of each gate over time and combine them robustly instead of trusting any single frame.

1) Associating sightings with gates: Each new world-frame estimate is assigned to the nearest known gate if it lies within 0.6 m of that gate's current center. Otherwise it starts a new gate. If two gates drift to within 0.6 m of each other, they are merged. We do not hard-code the number of gates, so the course is discovered as the drone flies. A gate is only used for planning after it has gathered at least five sightings.

2) Prioritizing close sightings: Each gate keeps at most its 15 closest sightings. Once five or more of these were taken within 1.5 m of the gate, only the close sightings are used in the fit. As a result, the estimate becomes more precise as the drone approaches instead of being pulled toward the biased long-range measurements.

3) RANSAC: We represent each sighting as a 6-vector $\mathbf{x}_k = [\mathbf{c}_k, \mathbf{n}_k]$ and fit a consensus pose with RANSAC [8]. In each iteration, a random subset of sightings is averaged to produce a candidate pose $(\bar{\mathbf{c}}, \bar{\mathbf{n}})$. Every sighting is then scored against the candidate. Position error is measured in meters and orientation error in radians, so the two cannot share one threshold. We therefore normalize each by its own tolerance and take the larger value:

$$e_k = \max\left(\frac{\|\mathbf{c}_k - \bar{\mathbf{c}}\|}{\tau_p}, \frac{\arccos|\mathbf{n}_k \cdot \bar{\mathbf{n}}|}{\tau_\theta}\right) \tag{4}$$

A sighting counts as an inlier when $e_k < 1$, meaning it is within 10 cm and 10° of the candidate. We keep the candidate with the most inliers, breaking ties by the smallest total inlier error, and require at least eight inliers for a candidate to be accepted. The final pose is recomputed from all inliers of the best candidate. If no candidate reaches consensus, the previous estimate is kept. Each random subset holds ten sightings, so a gate with fewer than ten sightings uses the plain average of all of them instead.

4) Averaging orientations: Taking the arithmetic mean of the normals does not work. The result is not a unit vector, and if any normal has the wrong sign, opposing normals partly cancel. Instead, we take the mean normal to be the principal eigenvector of the scatter matrix

$$\mathbf{S} = \sum_k \mathbf{n}_k \mathbf{n}_k^\top, \tag{5}$$

which is unaffected by the sign of each $\mathbf{n}_k$. We then choose the sign that agrees with the majority of the sightings. Gate centers are averaged with an ordinary mean.

V. Policy

The policy runs at 15 Hz. On each frame it runs the window detector, updates the world-frame gate map, and returns a body-frame velocity command $[v_x, v_y, v_z]$ with a yaw rate $\dot{\psi}$. The policy flies to the first confirmed gate it has not yet passed, using the map built from the detections.

A. State Machine

The policy is a finite state machine with five modes: `Search`, `Plan`, `Fly`, `Through`, and `Hover` (Figure 3).

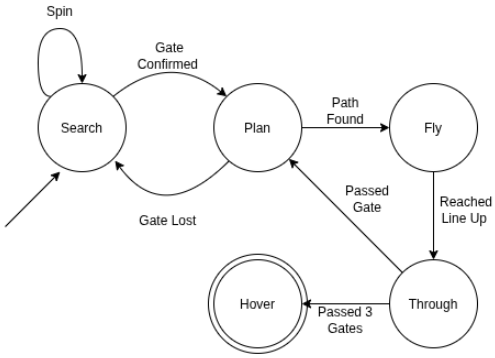


Fig. 3: Policy state machine.

1) Search: The drone holds its position and yaws at a constant 0.3 rad/s until a confirmed gate is in the map, and then switches to `Plan`. If the nearest detection is cut off at the image edge, PnP cannot be run on it. In that case, instead of yawing at a constant rate, the drone yaws and climbs toward the window for up to 60 frames until the window is fully in view.

2) Plan: If the straight line to the line-up point is collision free, the drone flies it at 1.0 m/s. Otherwise we plan with RRT* [9] in an obstacle map built from the confirmed gates. Each gate's frame is added as an obstacle at its estimated pose, inflated by a 0.10 m safety margin. The flight area's walls, floor and ceiling bound the map. We run RRT* search with a step size of 0.125 m and at most 750 iterations. Once the tree first reaches the goal, the search runs for up to 250 more iterations with the goal bias dropped from 15% to zero, so that rewiring can shorten the path. The lowest-cost path to the goal is then kept.

The raw RRT* path zig-zags because of its random samples, so we simplify it by shortcutting. Starting from the first waypoint, we skip ahead to the furthest waypoint that can still be reached in a collision-free straight line, keep that point, and repeat from it until we reach the goal. This removes every waypoint that is not needed to get around an obstacle, so the path becomes a few straight segments that bend only around the gates. The simplified path is then smoothed into a piecewise quintic polynomial trajectory for the controller to follow.

3) Fly: The drone follows the planned path while the gate estimate continues to improve. The path is replanned without stopping if the line-up point moves by more than 15 cm or a newly mapped gate blocks the remaining path. If the target gate is lost, or a cut-off window appears more than 1.5 m from the target, the drone stops and returns to `Search`. The second case prevents it from flying into a gate it has only partly seen. When the drone reaches the line-up point, it moves to `Through`.

4) Through: At this point, the drone commits to the gate. Close up, the gate fills the image and then leaves it, so detections are no longer useful. A fixed straight path is built from the current position through the gate center to an exit

point $\mathbf{c} + 0.4\,\mathbf{n}$, and is flown at 0.8 m/s. This path is not replanned. The drone still yaws toward the next gate in the map, so it collects sightings of that gate while passing through. On reaching the exit point, the gate is marked as passed. The drone then returns to `Plan`, or to `Hover` once all three gates have been passed.

5) Hover: After the last gate, the drone holds its position and yaws at 0.3 rad/s.

6) Unconfirmed gates: A gate seen fewer than five times may lie between the drone and its target. Skipping such a gate risks flying into it. Before planning, the drone therefore turns toward any unconfirmed gate ahead of the target for up to 45 frames to give it a chance to be confirmed. A gate that is still unconfirmed after that is treated as a false detection.

B. Control Law

The translational controller is the pure pursuit [10] position loop from our previous project, with retuned parameters. As before, the drone closes its own attitude and velocity loops in firmware, so the policy only sends a velocity command at 15 Hz. New to this project is a yaw controller, which keeps the camera pointed at the gate.

1) Target Selection: Each planned path is sampled every 5 cm into waypoints, each with a planned velocity. The policy keeps an index i into the waypoint list and advances it while the next waypoint is closer to the drone than the current one. From there it walks forward along the path by a lookahead distance of $L = 0.15$ m to get the target $\mathbf{p}_{\text{target}}$. If the lookahead runs past the end of the path, the last waypoint is used. The lookahead is half that of our previous project, so the drone cuts corners less on the short legs between gates.

2) Translation: The commanded world-frame velocity is the planned velocity at the closest waypoint, used as a feedforward term, plus a proportional correction toward the path:

$$\mathbf{v}_{\text{cmd}} = \mathbf{v}_{\text{ff}} + K_p\,\mathbf{e}_{\perp}, \qquad \mathbf{e}_{\perp} = \mathbf{e} - (\mathbf{e}\cdot\hat{\mathbf{u}})\,\hat{\mathbf{u}}, \tag{6}$$

where $\mathbf{e} = \mathbf{p}_{\text{target}} - \mathbf{p}$ and $\hat{\mathbf{u}} = \mathbf{v}_{\text{ff}}/\|\mathbf{v}_{\text{ff}}\|$. Removing the along-path component means the feedback only pulls the drone back onto the path, while the feedforward alone sets its speed. At the end of the path the planned velocity is zero, so the full error is used and the drone settles on the last waypoint. We kept $K_p = 2.5$ and capped $\|\mathbf{v}_{\text{cmd}}\|$ at 1.25 m/s. A path is complete once the drone is at its last waypoint and within 0.15 m of it.

3) Yaw: Our previous policy commanded zero yaw rate. Here the camera has to stay on the gate, so the yaw rate is a saturated proportional controller,

$$\dot{\psi}_{\text{cmd}} = \text{clip}\Big(2.0\,\tilde{\psi},\,-1,\,1\Big) \text{ rad/s}. \tag{7}$$

More than 1 m from the gate, the desired heading is the bearing to its center. Closer than that, the bearing swings quickly with small position changes, so the drone faces along the gate normal instead. Finally, $\mathbf{v}_{\text{cmd}}$ is rotated into the body frame by the drone's yaw ψ,

$$\mathbf{v}_{\text{body}} = \begin{pmatrix} \cos\psi & \sin\psi & 0 \\ -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \mathbf{v}_{\text{cmd}}. \tag{8}$$

TABLE II: Detector Performance at Score 0.5 and IoU 0.5

	Synthetic test	Splat
Images	2,458	206
Scored windows	3,932	223
Precision	0.990	0.967
Recall	0.996	0.933
Images without a window, falsely detected	0 of 239	1 of 66
Corner error, mean (px)	1.24	5.59*
Corner error, median (px)	0.81	4.37*

*Upper bound, includes error in the splat ground truth.

TABLE III: Recall and Mean Corner Error by Window Distance

	Synthetic test		Splat	
Distance (m)	Recall	Error (px)	Recall	Error (px)
0.5 to 1.0	0.993	2.98	0.80	10.6
1.0 to 1.5	0.998	2.02	0.90	8.3
1.5 to 2.0	0.994	1.52	1.00	5.5
2.0 to 3.0	0.997	1.04	1.00	3.7
3.0 to 4.0	0.996	0.82	1.00	2.7

VI. Results

A. Window Detection

The detector is evaluated on two sets. The first is the synthetic test split from Section II. The second is a set of 206 views rendered from the Gaussian splat of the training course, which the network never saw. They are the six course views of the starter pack plus 200 random camera poses over the flight area, mostly looking towards a window. The ground truth for these views is projected from the window poses in the scene files, which are provided for checking the detector and are never read by the flight code. A window is scored when its printed side faces the camera, it is 0.5 to 4 m away and at least 20% of its print is visible, the same rules as the training data. Detections of other windows, for example a window seen from behind, are neither counted as right nor as wrong.

A detection is kept when its score is above 0.5 and counts as correct when its box overlaps a ground truth box with an intersection over union of at least 0.5. Corner error is the pixel distance between the predicted and true corners of the opening, for every corner inside the image. The results are in Tables II and III, and predictions on the splat are shown in Fig. 4.

On the synthetic test split the detector finds nearly every window and never fires on an empty image. Its corners are within about a pixel of the truth, and corners hidden behind a nearer window are placed about twice as far off as visible ones. Error grows as windows get closer because the same relative error covers more pixels.

On the splat views, precision and recall stay above 0.93, so the detector transfers from our synthetic images to the splat without ever having seen it. All 15 missed windows have at most one corner of the opening inside the image. These are windows so close and so cut off that only a strip of the frame is visible, like the last two images in Fig. 4. Every window with at least two corners of the opening in view is found, which covers all 99 windows whose opening is fully in view. In six of the misses the network does detect the window, but its box overlaps the truth by less than the 0.5 needed to count.

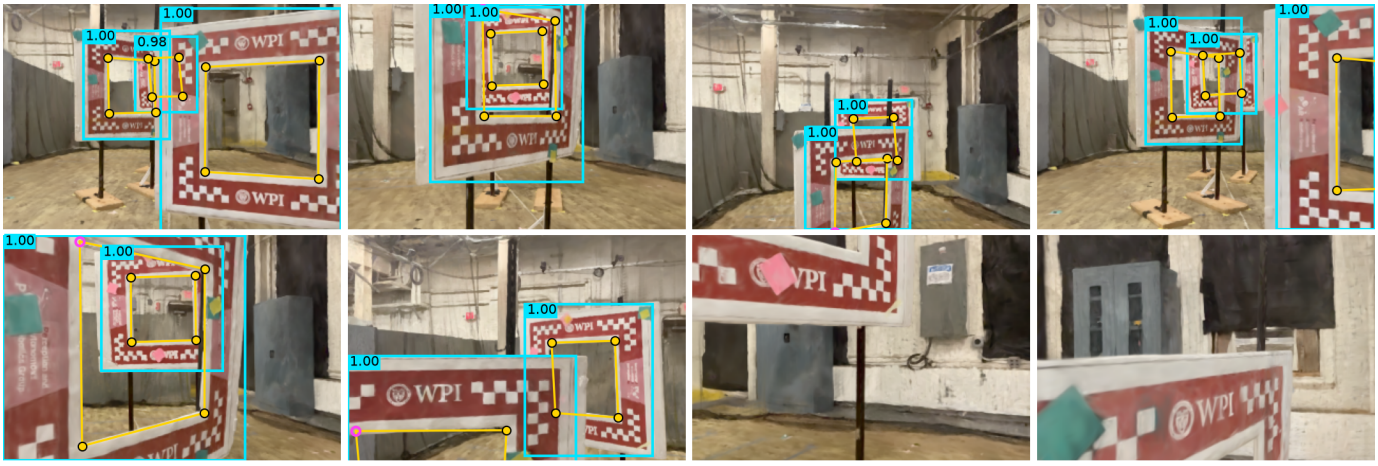


Fig. 4: Network predictions on splat renders of the training course, which were never used for training. Cyan boxes are detections with their scores. Filled yellow points are corners the network rates as inside the image and hollow magenta points are corners it rates as outside. The top row shows the start of the course with all three windows, a window seen through another and two far windows. The bottom row shows close and oblique windows and two misses, where only a strip of the frame is in view.

B. Flight Outcomes

Table IV lists the eight runs from our second lab session, all flown with the final policy. Four of the eight runs completed the course. Every run passed the first two gates, and every failure occurred on the approach to the third gate.

TABLE IV: HIL runs on `a3_train`, second lab session. Times are measured from the start of the run.

Run	Gate 1 (s)	Gate 2 (s)	Gate 3 (s)	Duration (s)	Outcome
2	2.60	5.47	–	10.60	hit, G3
3	2.53	5.27	–	8.53	hit, G3
4	2.60	5.47	9.20	9.20	complete
5	2.67	5.27	–	8.47	hit, G3
6	2.60	5.27	9.00	9.00	complete
7	2.60	5.20	9.47	9.47	complete
8	2.67	5.67	–	8.60	hit, G3
9	2.60	5.07	9.33	9.33	complete

The completed runs took 9.25 ± 0.20 s. The first two gates were passed at very consistent times, about 2.6 s and 5.3 s, which shows that detection, pose estimation, and planning behaved repeatably across runs. Nearly all of the variation in lap time comes from the final leg.

C. Failure Analysis

The third gate sits higher than the drone's position after the second gate, so this leg requires a climb of about 0.5 m. In every run the drone overshot the climb, reaching about 1.3 m, and then descended toward the gate. In the completed runs it crossed the gate plane at a height of 1.14–1.23 m. Three of the four collisions (runs 3, 5 and 8) occurred near the gate plane at a height of only 1.00–1.05 m, with clearances of 14 to 46 mm. In these runs the drone was still descending as it reached the gate, so it struck the bottom of the opening. In run 2 the drone collided at 1.60 m, too high rather than too low. The onboard controller reported that its integrator saturated during the climb in all four failed runs. Among the completed runs, the warning appeared only in run 9, for four ticks between the first and second gates. These observations suggest that the drone's vertical response lags the commanded altitude change during the final leg, so the `Through` phase began before the drone had settled at the correct height.

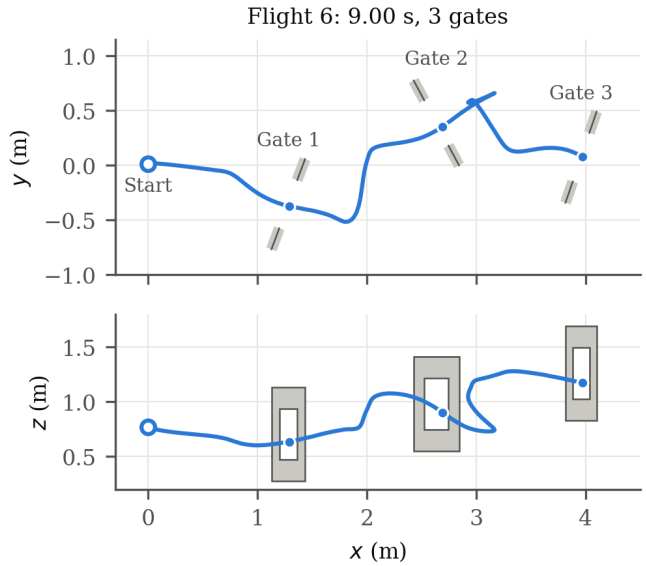


Fig. 5: Flight 6, top (upper) and side (lower) views. The gate boards are gray with the opening cut out. Filled dots mark where each gate was passed.

D. Timing

The policy fit well within the 66 ms frame budget on the lab PC. Across the eight runs, the median time per control

tick was 18–20 ms, and the 95th percentile was below 23 ms. Rendering took a median of about 8.5 ms and the policy, including detector inference, about 8.1 ms.

VII. Conclusion

This project combined a learned window detector, PnP-based pose estimation, RANSAC filtering, an RRT* planner, and a pure pursuit controller into a policy that flew a drone through three windows without prior knowledge of their locations. In hardware-in-the-loop flights, the drone completed the course in 9.25 ± 0.20 s, taking about 3 s per gate to detect it, estimate its pose, plan, and fly through. A control tick took a median of about 20 ms, leaving time to spare in every cycle for better algorithms. Four of eight flights completed the course. All four failures came on the climb to the highest gate, where the drone's altitude lagged the command. Waiting for the altitude to settle before committing to the gate is the most direct fix.

The pipeline was built to generalize to more fly-through tasks. After PnP, every later stage, including tracking, filtering, planning, and the pass-through path, works only on each opening's center and normal, and never assumes how many gates there are or where they are, only that they are flown in order of increasing x. A new opening type, such as a different size, a rectangle, or a circle, therefore only requires training the detector on it and supplying its keypoint geometry. With a class reported for each detection, a single course could mix several opening types. Future courses can be flown by using various detectors and describing the new openings, without rewriting the planner, controller, or state machine.

Acknowledgment

The authors would like to thank Prof. Nitin J. Sanket and TA Chaitanya Gaddipati for the opportunity to implement this project as a part of the RBE595-F02 Hands-on Autonomous Aerial Robotics course at WPI, and for the guidance that was provided along the way.

References

[1] Blender Online Community, "Blender, a 3D modelling and rendering package," https://www.blender.org, Blender Foundation, Amsterdam.

[2] G. Jocher, J. Qiu, and A. Chaurasia, "Ultralytics YOLO," https://github.com/ultralytics/ultralytics, 2023.

[3] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.

[4] I. E. Sutherland and G. W. Hodgman, "Reentrant polygon clipping," *Communications of the ACM*, vol. 17, no. 1, pp. 32–42, 1974.

[5] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, "Ultralytics YOLO26: Unified real-time end-to-end vision models," 2026. [Online]. Available: https://arxiv.org/abs/2606.03748

[6] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.

[7] T. Collins and A. Bartoli, "Infinitesimal plane-based pose estimation," *International Journal of Computer Vision*, vol. 109, no. 3, pp. 252–286, 2014.

[8] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[9] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.

[10] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie Mellon University, Robotics Institute, Pittsburgh, PA, Tech. Rep. CMU-RI-TR-92-01, 1992.